Project 2 - Master Control Program

Oliver Boorstein
University of Oregon

May 13, 2026

1 Introduction

This project implements a small Master Control Program in C. It reads a workload file, forks a child process for each command, controls when each child starts, and schedules the processes in a round-robin style using signals. The final part also prints process information from `/proc`. Compared to Project 1, this project was much more about process coordination than string parsing.

2 Background

The main idea behind the project is that the MCP acts like a very small operating system scheduler. The child processes do the actual workload, but the parent decides when they are allowed to run. This required a different mindset than Project 1. In Project 1, once a syscall succeeded, the command was mostly done. Here, a correct syscall could still be in the wrong order, especially with signals. A child might exit before the alarm fires, or the alarm might fire while the MCP is also receiving `SIGCHLD`.

3 Implementation

The first version launches every command from the input file. It uses the same basic tokenization approach as Project 1, splitting lines into commands and arguments, then calling `fork` and `execvp`. The parent stores each PID so it can wait for every child at the end.

Part 2 adds control before execution. The parent blocks `SIGUSR1` before forking, and each child waits for that signal before calling `execvp`. This avoids the race where the parent sends a start signal before the child is ready.

```
sigemptyset(&set);
sigaddset(&set, SIGUSR1);
sigprocmask(SIG_BLOCK, &set, &oldset);

if (pid == 0) {
  int sig;
  sigwait(&set, &sig);
  sigprocmask(SIG_SETMASK, &oldset, NULL);
  execvp(args.list[0], args.list);
}
```

After that, the MCP uses `SIGSTOP` and `SIGCONT` to suspend and resume workload processes. Part 3 replaces the simple PID array with a process list that tracks whether each process is `NEVER_RUN`, `RUNNING`, `STOPPED`, or `EXITED`. This status field ended up being important because a process that has never run needs `SIGUSR1`, while a stopped process needs `SIGCONT`. Treating both cases the same would either skip exec entirely or signal the wrong state.

The round-robin selection scans forward from the current process and skips exited entries. This part was one of the main challenges because the MCP has to handle both time-slice expiration and children exiting early. Using `waitpid(-1, &status, WNOHANG)` was important because a normal blocking wait would stop the MCP from scheduling the other processes. The alarm handler and

child handler only set `volatile sig_atomic_t` flags, and the main loop does the real work after `sigsuspend` returns. This made the scheduler cleaner because signal handlers only record that something happened, while the normal control flow decides whether to stop, continue, start, or reap a process.

Part 4 adds `/proc` reporting. I read `/proc/<pid>/status` for process state, memory usage, and context switches, and `/proc/<pid>/io` for read/write syscall counts and bytes. Formatting this cleanly was surprisingly annoying because processes may exit while the MCP is trying to read their files. I handled that by marking each source as available or unavailable and printing dashes or a short note when the file is gone. I also kept the table limited to values that were useful for understanding scheduling and resource use instead of dumping every field from `/proc`.

4 Performance

Performance is good for the assignment scale. The scheduler sleeps on `sigsuspend` and wakes from `SIGALRM` or `SIGCHLD`, so it is not wasting CPU by polling. Round-robin selection is linear in the number of children, but the workloads are small enough that this is fine. The `/proc` reads add overhead each cycle because the MCP opens and parses files repeatedly, but they make the output much more useful. The two-second time slice also makes the output easy to follow while still showing real scheduling behavior.

5 Conclusion

Overall, this was much harder than Project 1. The basic workload launching was straightforward, but signal timing, alarms, round-robin process selection, child reaping, and `/proc` formatting made the project more interesting. The final MCP can start processes only when scheduled, stop and continue them across time slices, skip exited processes, and print useful process information each cycle.